LAPORAN TUGAS PRAKTIKUM PERTEMUAN 11
Membuat form operasi aritmatika

1

untuk memenuhi tugas pada mata kuliah:

Pemrograman Berbasis Objek

Dosen pengampu:

Endang Purnama Giri S.Kom., M.Kom.



Muhamad Ilham Nurhakim (J0403251004)
Haidar Hafizh Izzuddin (J0403251031)
Ihsan Hamizan (J0403251051)

Program Studi Teknologi Rekayasa Perangkat Lunak
Sekolah Vokasi
Institut Pertanian Bogor
2026

Source Code dan Link Github

https://github.com/i-zzean/PBO_Perhitungan

DemoApplication.java

Code Blame 13 lines (9 loc) · 305 Bytes Raw

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

- **package & import (Baris 1–4):** Menentukan lokasi direktori file dan memanggil dependensi Spring Boot yang dibutuhkan.
- **@SpringBootApplication (Baris 6):** Anotasi utama yang mengaktifkan konfigurasi otomatis (*auto-configuration*) dan pencarian komponen proyek secara otomatis.
- **public class DemoApplication (Baris 7):** Deklarasi nama kelas utama aplikasi.
- **main method (Baris 9–10):** Fungsi yang pertama kali dieksekusi saat program dijalankan. Di dalamnya, perintah SpringApplication.run(...) memuat seluruh konfigurasi Spring sekaligus menyalakan *embedded web server* (seperti Tomcat) agar aplikasi langsung aktif.

Perhitungan.java

Code Blame 60 lines (40 loc) · 1.12 KB Raw

```java
package com.example.demo;

import jakarta.persistence.*;


@Entity
@Table(name = "perhitungan")
public class Perhitungan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "idoperasi")
    private Long idOperasi;

    @Column(name = "bilangan1")
    private float bilangan1;

    @Column(name = "bilangan2")
    private float bilangan2;

    @Column(name = "operator")
    private char operator;


    public Long getIdOperasi(){
        return idOperasi;
    }

    public void setIdOperasi(Long idOperasi){
        this.idOperasi = idOperasi;
    }

    public float getBilangan1() {
        return bilangan1;
    }

```

```
37        public void setBilangan1(float bilangan1) {
38            this.bilangan1 = bilangan1;
39        }
40
41
42        public float getBilangan2() {
43            return bilangan2;
44        }
45
46        public void setBilangan2(float bilangan2) {
47            this.bilangan2 = bilangan2;
48        }
49
50        public void setOperator(char operator){
51            this.operator = operator;
52        }
53
54        public char getOperator(){
55            return operator;
56        }
57
58
59
60    }
```

- **Pemetakan Tabel (@Entity, @Table):**
  - @Entity menandai bahwa kelas Perhitungan adalah tabel dalam database.
  - @Table(name = "perhitungan") mengarahkan kelas ini ke tabel database bernama perhitungan.
- **Atribut & Kolom (@Column):**
  - idOperasi (Long): Berfungsi sebagai *Primary Key* (@Id) dengan nilai otomatis/auto-increment (GenerationType.IDENTITY).
  - bilangan1 (float): Menyimpan angka pertama.
  - bilangan2 (float): Menyimpan angka kedua.
  - operator (char): Menyimpan simbol operasi matematika (misalnya +, -, *, /).
- **Getter & Setter (Baris 25–56):**
  - Berisi metode standar untuk mengambil (*getter*) dan mengisi/mengubah (*setter*) nilai dari setiap variabel privat di atas

PerhitunganController.java

Code Blame 56 lines (39 loc) · 1.93 KB Raw

```java
package com.example.demo;

import java.io.IOException;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.ui.Model;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.PostMapping;


@Controller
public class PerhitunganController {

    @Autowired
    private PerhitunganRepository PerhitunganRepository;

    @GetMapping("/perhitungan")
    public String tampilkanHalamanPerhitungan(Model model) {
        model.addAttribute("listRiwayatOperasi", PerhitunganRepository.findAll(Sort.by(Sort.Direction.ASC, "idOperasi")));
        model.addAttribute("perhitungan", new Perhitungan()); // Objek kosong untuk form input
        return "perhitungan";
    }

    @PostMapping("/perhitungan/simpan")
    public String postMethodName(@ModelAttribute Perhitungan perhitungan) throws IOException {
        if (perhitungan.getIdOperasi() != null) {
            Perhitungan perhitunganLama = PerhitunganRepository.findById(perhitungan.getIdOperasi()).orElseThrow(() -> new RuntimeException("Data tidak ditemukan"));
            perhitunganLama.setBilangan1(perhitungan.getBilangan1());
            perhitunganLama.setBilangan2(perhitungan.getBilangan2());
            perhitunganLama.setOperator(perhitungan.getOperator());

            PerhitunganRepository.save(perhitunganLama);

        } else {
            PerhitunganRepository.save(perhitungan);
        }

        return "redirect:/perhitungan";

    }


    @PostMapping("/perhitungan/delete/{id}")
    public String hapusData(@PathVariable Long id) {

        PerhitunganRepository.deleteById(id);

        return "redirect:/perhitungan";
    }


}
```

- **Injeksi Repository (@Autowired):**
  - Memanggil PerhitunganRepository untuk melakukan operasi CRUD ke database.
- **Menampilkan Halaman (@GetMapping("/perhitungan") — Baris 21–26):**
  - Mengambil seluruh daftar riwayat perhitungan dari database yang diurutkan berdasarkan idOperasi secara menaik (*ascending*).
  - Menyiapkan objek kosong Perhitungan untuk menampung input form.
  - Menampilkan halaman HTML perhitungan.
- **Menyimpan / Perbarui Data (@PostMapping("/perhitungan/simpan") — Baris 28–44):**
  - **Update:** Jika idOperasi sudah ada, kode akan mencari data lama di database lalu memperbarui nilainya (bilangan1, bilangan2, operator).
  - **Create:** Jika idOperasi kosong/null, data perhitungan baru akan langsung disimpan ke database.
  - Setelah simpan, halaman dialihkan kembali (*redirect*) ke /perhitungan.
- **Menghapus Data (@PostMapping("/perhitungan/delete/{id}") — Baris 47–53):**

- Menghapus data perhitungan berdasarkan id yang dikirim dari URL parameter.
- Mengalihkan (*redirect*) kembali ke halaman /perhitungan.

PerhitunganRepository.java

Code Blame 9 lines (7 loc) · 320 Bytes Raw

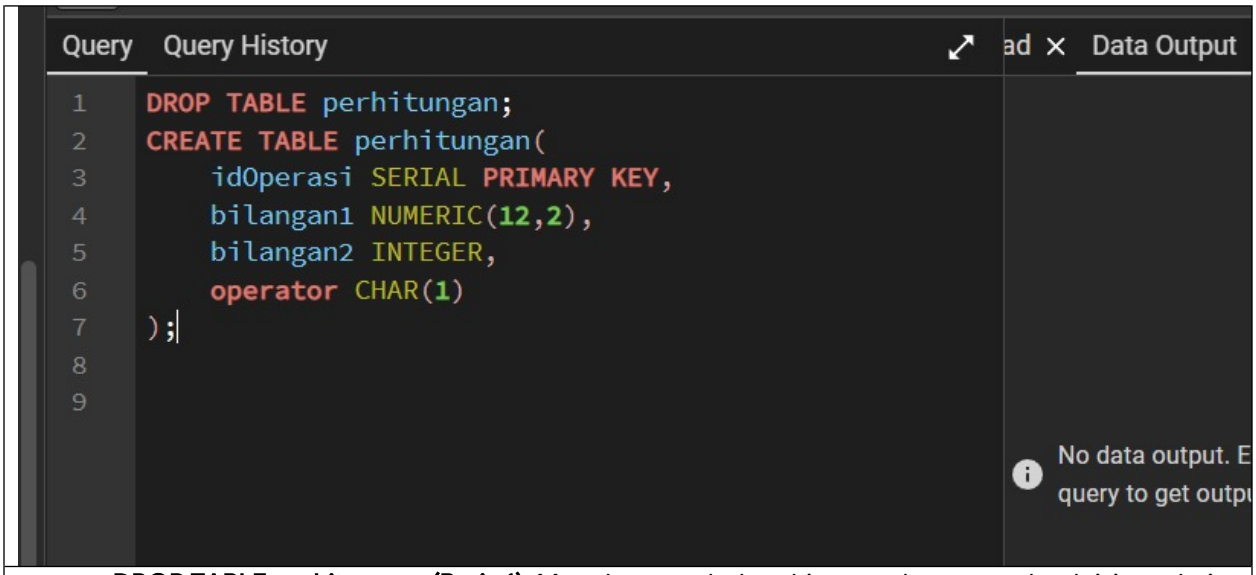
```java
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PerhitunganRepository extends JpaRepository<Perhitungan, Long> {
    // Menyediakan method findAll(), save(), findById(), dll secara otomatis
}
```

- **@Repository (Baris 6):** Anotasi untuk menandai bahwa interface ini berfungsi sebagai komponen penyimpanan/akses data dalam arsitektur Spring.
- **extends JpaRepository<Perhitungan, Long> (Baris 7):** Menurunkan seluruh fitur bawaan JpaRepository untuk entity Perhitungan dengan tipe *Primary Key* Long.
- **Operasi Otomatis (Baris 8):** Mengaktifkan metode CRUD dasar (seperti findAll(), findById(), save(), deleteById()) secara otomatis tanpa perlu menulis query SQL secara manual.

Query Database



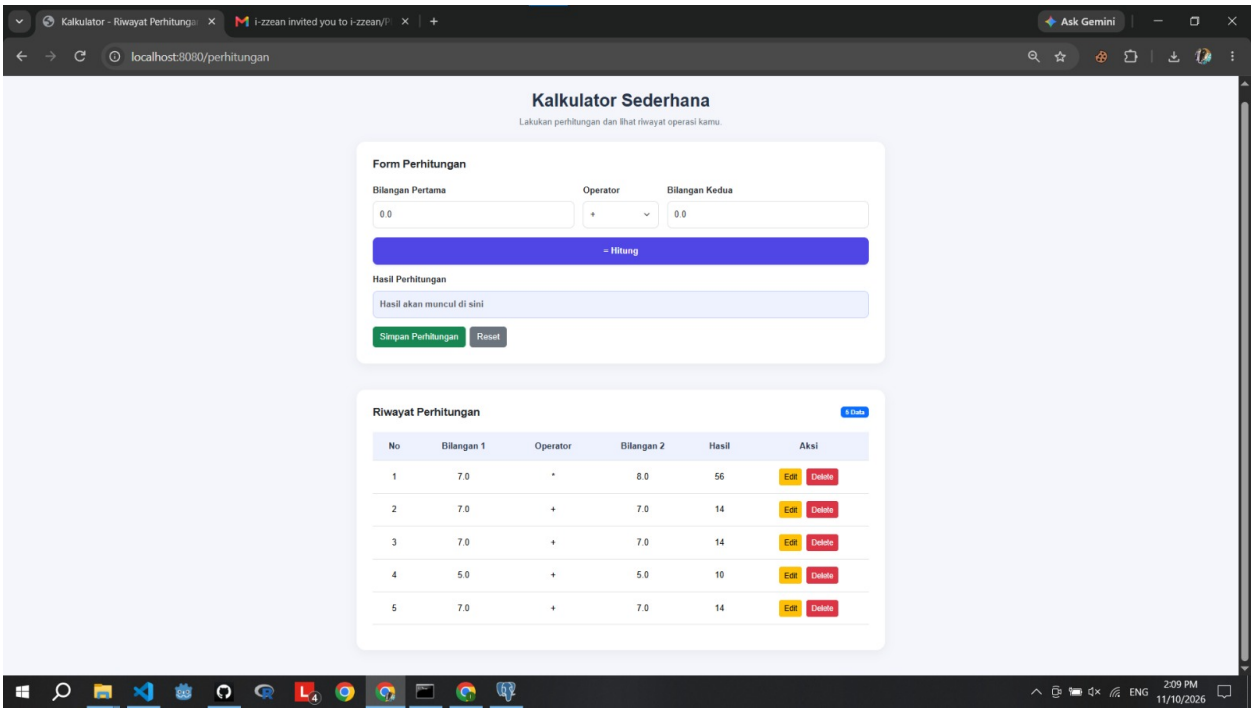
- **DROP TABLE perhitungan; (Baris 1):** Menghapus tabel perhitungan beserta seluruh isinya dari database jika tabel tersebut sudah ada sebelumnya.
- **CREATE TABLE perhitungan (...) (Baris 2–7):** Membuat tabel baru bernama perhitungan dengan struktur kolom sebagai berikut:
- **idOperasi SERIAL PRIMARY KEY:** Kolom ID yang berfungsi sebagai *Primary Key* dan nilainya bertambah secara otomatis (*auto-increment*).
- **bilangan1 NUMERIC(12,2):** Kolom untuk menyimpan angka desimal hingga 12 digit total dan 2 angka di belakang koma.
- **bilangan2 INTEGER:** Kolom untuk menyimpan angka bulat.
- **operator CHAR(1):** Kolom untuk menyimpan 1 karakter simbol operasi (seperti +, -, *, /).



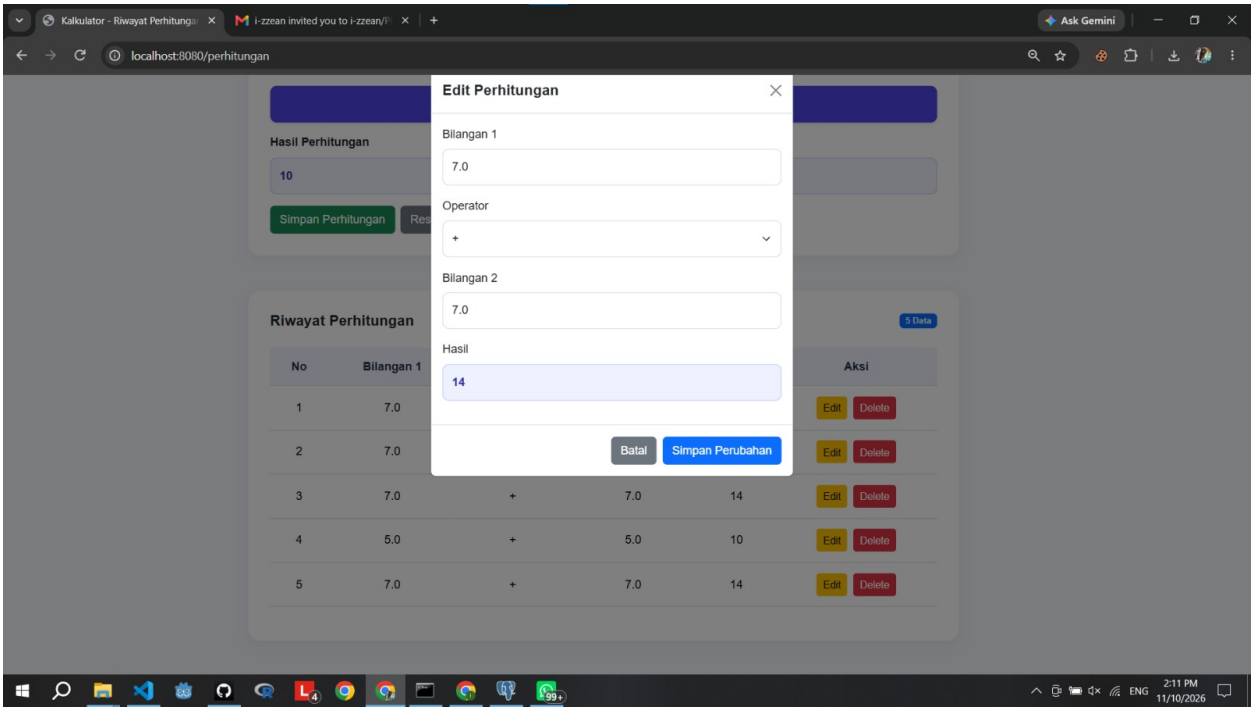
	idoperasi [PK] bigint	bilangan1 real	bilangan2 real	operator character (1)
1	3	7	7	+
2	2	7	8	*
3	4	7	7	+
4	7	5	5	+

Hasil ketika sudah di Run

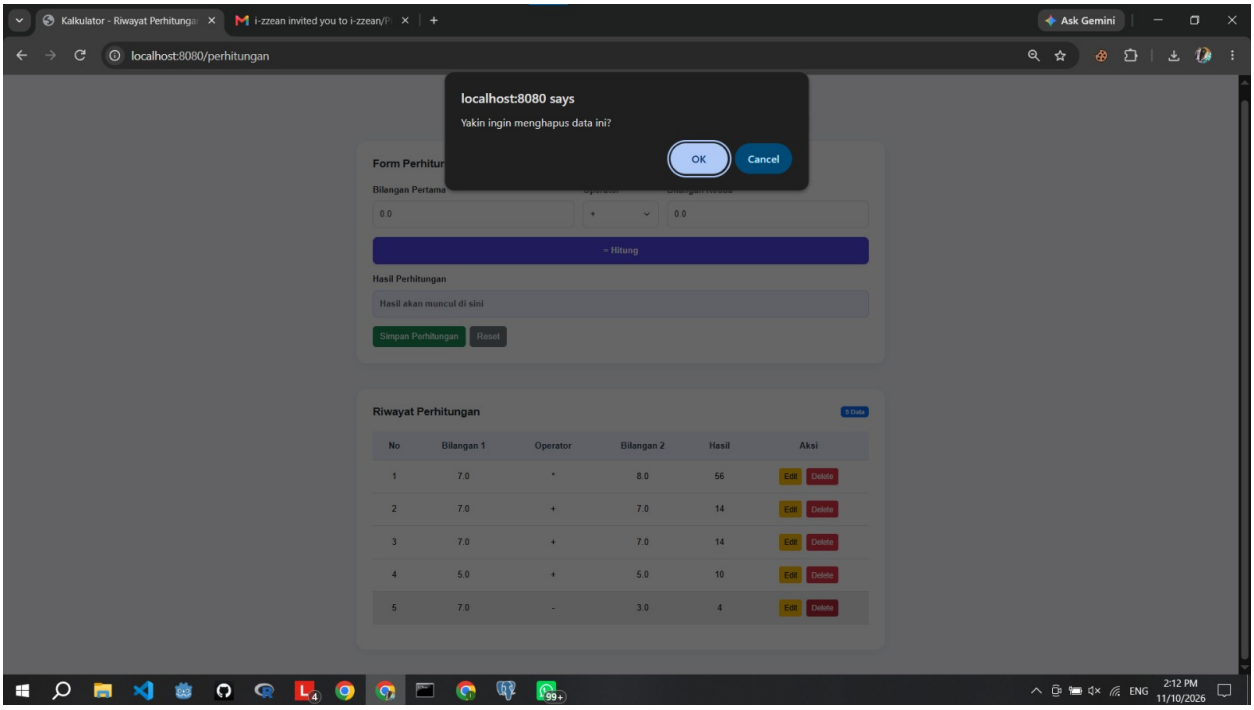
Tampilan Awal



Edit Perhitungan



Hapus Perhitungan



Perhitungan belum tersimpan

